

INFORME TÉCNICO

Análisis Algorítmico y Arquitectura del Sistema

Sistema de Gestión de Tienda de Abarrotes

Presentación T2 · Duración estimada: 10–12 minutos

1. Problema y Objetivos

Una tienda de abarrotes gestiona ventas, stock y caja de forma manual, lo que genera errores de inventario, pérdida de tiempo en cálculos y falta de trazabilidad. El sistema desarrollado automatiza estos procesos mediante una aplicación de escritorio en Python con base de datos local SQLite.

Objetivos principales: (1) Registrar ventas con validación de stock y cálculo de vuelto; (2) Mantener inventario actualizado en tiempo real; (3) Controlar movimientos de caja; (4) Garantizar integridad de datos mediante transacciones.

Se recomienda incluir el diagrama de Ishikawa (evidencia de la semana 4) para mostrar las causas raíz del problema: errores humanos, falta de registros históricos y ausencia de control de stock.

2. Arquitectura del Sistema

El sistema sigue una arquitectura en capas con separación clara de responsabilidades:

Capa	Archivo	Responsabilidad
Presentación	main.py	Interfaz gráfica con Tkinter: formularios, tablas, eventos de usuario y validaciones de UI.
Lógica / Datos	db.py	Operaciones CRUD, transacciones, consultas SQL y reglas de negocio (stock, caja, ventas).
Persistencia	SQLite	Base de datos local con índices, claves foráneas y PRAGMA foreign_keys = ON.

Esta separación permite modificar la interfaz sin alterar la lógica de negocio, y facilita pruebas unitarias sobre las funciones de db.py de forma independiente.

3. Estructuras de Datos (Semana 1)

3.1 Registros

Se utiliza `sqlite3.Row` con `conn.row_factory`. Cada fila se comporta como un registro con acceso por nombre de campo: `r['name']`, `r['price']`, etc. Equivale a un registro estructurado sin necesidad de clases adicionales.

3.2 Diccionarios y Listas

El carrito de compras es una lista de diccionarios: `self.cart = [{'product_id': ..., 'name': ..., 'quantity': ..., 'price': ...}, ...]`. Permite agregar, buscar y actualizar ítems de forma flexible. Las tablas de la interfaz se redibujan recorriendo estas colecciones.

3.3 Cadenas

Formato monetario con f-strings: `f"$ {v:,.2f}"`. Normalización con `.strip()` y `.upper()`. Números de venta generados como `f"V{today}-{n:04d}"` (ej. `V20261002-0001`).

4. Algoritmos de Búsqueda y Ordenamiento (Semana 2)

4.1 Búsqueda lineal y fuerza bruta

- list_products(search):** usa `LIKE '%texto%'` sobre código, nombre y categoría. El comodín inicial impide el uso de índices → escaneo completo $O(n)$.
- add_selected_to_cart:** `next((i for i in self.cart if i['product_id']==pid), None)` realiza búsqueda lineal en el carrito $O(k)$, donde k = cantidad de ítems.
- refresh_cart / refresh_products:** recorren todos los elementos para redibujar las tablas de la interfaz.

4.2 Ordenamiento

Las consultas usan `ORDER BY name COLLATE NOCASE`, que ordena sin distinguir mayúsculas/minúsculas. SQLite implementa algoritmos eficientes (típicamente mezcla o similar, $O(n \log n)$), en contraste con burbuja $O(n^2)$ del sílabo. El motor de base de datos garantiza un rendimiento superior al de implementaciones manuales en Python puro.

5. Complejidad de las Funciones Clave

Función	Qué hace	Complejidad
<code>list_products</code>	Escaneo completo + <code>ORDER BY name</code>	$O(n \log n)$ por el ordenamiento
<code>get_product</code> (por id)	Búsqueda por clave primaria	$O(\log n)$ — árbol B de SQLite
<code>code UNIQUE</code>	Índice sobre código de producto	Validación de duplicados $O(\log n)$
<code>add_selected_to_cart</code>	Búsqueda lineal en el carrito	$O(k)$, k = ítems del carrito
<code>create_sale</code>	Valida y guarda k ítems con búsquedas por clave	$O(k \log n)$
<code>dashboard()</code>	COUNT y SUM sobre productos	$O(n)$

Punto de mejora destacable: el total del carrito se recalcula con `sum( . . . )` en `refresh_cart`, `payment_changed` y `update_change` ($O(k)$ cada vez). Mantener un total acumulado reduciría estas operaciones a $O(1)$.

6. Flujo Crítico: `create_sale`

Es el algoritmo más completo del sistema. Sirve como ejemplo de diseño algorítmico: entrada → validaciones → proceso → salida.

1. Validar que el carrito no esté vacío y calcular el total.
2. Si el pago es en efectivo, verificar `received ≥ total` y calcular el vuelto.
3. Generar el número de venta con `next_sale_number()`.
4. Por cada ítem: verificar existencia y stock suficiente.
5. Insertar la venta y luego los `sale_items`.
6. Descontar stock y registrar el movimiento de tipo SALIDA.
7. Registrar el INGRESO en caja.

Transacciones y validación (calidad de software)

Todo ocurre dentro de `with self.connect()` as `con`. Si cualquier paso falla, la transacción se revierte: venta, stock y caja permanecen consistentes. La validación es de doble capa: interfaz (`add_selected_to_cart`) y base de datos (`create_sale`, `adjust_stock` impide stock negativo). `PRAGMA foreign_keys = ON` y las claves foráneas protegen la integridad referencial. Los productos se desactivan (`active=0`) en lugar de borrarse, para no romper el historial de ventas ligado por FK.

7. Mejoras con Estrategias del Curso (Semanas 3–5)

El código actual no implementa aún las estrategias avanzadas del sílabo. Se proponen mejoras honestas y factibles:

Estrategia	Mejora aplicable
Algoritmo voraz (sem. 3)	Desglosar el vuelto en billetes/monedas (S/ 200, 100, 50, 20, 10, 5, 2, 1, 0.50, 0.20, 0.10) tomando siempre la denominación mayor posible.
Divide y vencerás / Búsqueda binaria (sem. 3–4)	Mantener productos ordenados por código en memoria y buscar con búsqueda binaria $O(\log n)$.
Recursividad (sem. 3)	Calcular el total de una lista de ítems o agrupar categorías de forma recursiva.
Programación dinámica (sem. 5)	Problema de la mochila: dado un presupuesto de reposición, elegir productos que maximicen la ganancia.
Algoritmos probabilistas (sem. 6)	Mencionar solo como trabajo futuro (opcional).

Recomendación: implementar el desglose de vuelto (voraz) y/o la búsqueda binaria. Mencionar el resto como trabajo futuro. Así se demuestra aplicación del curso sin forzar relaciones inexistentes.

8. Herramientas, Conclusiones y Recomendaciones

8.1 Stack tecnológico

- **Python + Tkinter:** interfaz de escritorio multiplataforma.
- **SQLite:** persistencia local, transacciones ACID, índices B-tree.
- **PyInstaller + build_windows.bat:** genera `MiTienda.exe`.
- **Git:** control de versiones (requerido en unidad II del sílabo).

- **IA:** apoyo en estructura inicial, depuración y documentación (verificar siempre el resultado).

8.2 Checklist de evidencias T2

- Informe con descripción de herramientas, procedimientos, conclusiones y recomendaciones.
- Producto funcionando (MiTienda.exe o Ejecutar_Tienda.bat).
- Certificado Python Essentials 1 (Cisco Networking Academy).
- Repositorio Git con varios commits (evidencia de control de versiones).
- Explicación breve del uso de herramientas de IA y de la verificación propia realizada.

8.3 Preguntas frecuentes y respuestas clave

- **¿Por qué SQLite y no una lista en memoria?** Persistencia, transacciones y búsquedas $O(\log n)$ con índices.
- **¿Complejidad de buscar un producto?** Con LIKE '%x%' $\rightarrow O(n)$; por código con índice $\rightarrow O(\log n)$.
- **¿Qué pasa si falla la luz a mitad de una venta?** La transacción no se confirma; no queda venta a medias.
- **¿Cómo escalar a miles de productos?** Índices, paginación y búsqueda por prefijo (LIKE 'x%').
- **¿Por qué desactivar en vez de borrar?** active=0 preserva el historial de ventas ligado por clave foránea.

8.4 Conclusión

El sistema resuelve el problema de gestión manual mediante una arquitectura limpia, estructuras de datos adecuadas (registros, listas, diccionarios) y algoritmos con complejidades justificadas. Las transacciones garantizan consistencia. Las mejoras propuestas (voraz para vuelto, búsqueda binaria) permiten demostrar dominio de las estrategias del curso de forma concreta y honestas. El producto es desplegable como ejecutable y cumple los requisitos de evidencias de la T2.